

Consider what happens to the value $f_a^{(r)}(k)$ as we flip a single bit k_i of the input value k , for various values of r . Let $k = \sum_{i=0}^{w-1} k_i 2^i$ and $g_a(k) = \sum_{j=0}^{w-1} b_j 2^j$ define the bit values k_i in the input (with k_0 the least-significant bit) and the bit values b_j in $g_a(k) = (2k^2 + ak) \bmod 2^w$ (where $g_a(k)$ is the value that, when its halves are swapped, becomes $f_a(k)$). Suppose that flipping a single bit k_i of the input k may cause any bit b_j of $g_a(k)$ to flip, for $j \geq i$. What is the least value of r for which flipping the value of any single bit k_i may cause *any* bit of the output $f_a^{(r)}(k)$ to flip? Explain.

Problems

11-1 Longest-probe bound for hashing

Suppose you are using an open-addressed hash table of size m to store $n \leq m/2$ items.

- a. Assuming independent uniform permutation hashing, show that for $i = 1, 2, \dots, n$, the probability is at most 2^{-p} that the i th insertion requires strictly more than p probes.
- b. Show that for $i = 1, 2, \dots, n$, the probability is $O(1/n^2)$ that the i th insertion requires more than $2 \lg n$ probes.
Let the random variable X_i denote the number of probes required by the i th insertion. You have shown in part (b) that $\Pr\{X_i > 2 \lg n\} = O(1/n^2)$. Let the random variable $X = \max \{X_i : 1 \leq i \leq n\}$ denote the maximum number of probes required by any of the n insertions.
- c. Show that $\Pr\{X > 2 \lg n\} = O(1/n)$.
- d. Show that the expected length $E[X]$ of the longest probe sequence is $O(\lg n)$.

11-2 Searching a static set

You are asked to implement a searchable set of n elements in which the keys are numbers. The set is static (no INSERT or DELETE operations), and the only operation required is SEARCH. You are given an arbitrary amount of time to preprocess the n elements so that SEARCH operations run quickly.

- a.** Show how to implement SEARCH in $O(\lg n)$ worst-case time using no extra storage beyond what is needed to store the elements of the set themselves.
- b.** Consider implementing the set by open-address hashing on m slots, and assume independent uniform permutation hashing. What is the minimum amount of extra storage $m - n$ required to make the average performance of an unsuccessful SEARCH operation be at least as good as the bound in part (a)? Your answer should be an asymptotic bound on $m - n$ in terms of n .

11-3 Slot-size bound for chaining

Given a hash table with n slots, with collisions resolved by chaining, suppose that n keys are inserted into the table. Each key is equally likely to be hashed to each slot. Let M be the maximum number of keys in any slot after all the keys have been inserted. Your mission is to prove an $O(\lg n / \lg \lg n)$ upper bound on $E[M]$, the expected value of M .

- a.** Argue that the probability Q_k that exactly k keys hash to a particular slot is given by

$$Q_k = \left(\frac{1}{n}\right)^k \left(1 - \frac{1}{n}\right)^{n-k} \binom{n}{k}.$$

- b.** Let P_k be the probability that $M = k$, that is, the probability that the slot containing the most keys contains k keys. Show that $P_k \leq nQ_k$.
- c.** Show that $Q_k < e^k/k^k$. *Hint:* Use Stirling's approximation, equation (3.25) on page 67.

d. Show that there exists a constant $c > 1$ such that $Q_{k_0} < 1/n^3$ for $k_0 = c \lg n / \lg \lg n$. Conclude that $P_k < 1/n^2$ for $k \geq k_0 = c \lg n / \lg \lg n$.

e. Argue that

$$E[M] \leq \Pr \left\{ M > \frac{c \lg n}{\lg \lg n} \right\} \cdot n + \Pr \left\{ M \leq \frac{c \lg n}{\lg \lg n} \right\} \cdot \frac{c \lg n}{\lg \lg n}.$$

Conclude that $E[M] = O(\lg n / \lg \lg n)$.

11-4 Hashing and authentication

Let $\mathcal{H}$ be a family of hash functions in which each hash function $h \in \mathcal{H}$ maps the universe U of keys to $\{0, 1, \dots, m-1\}$.

a. Show that if the family $\mathcal{H}$ of hash functions is 2-independent, then it is universal.

b. Suppose that the universe U is the set of n -tuples of values drawn from $\mathbb{Z}_p = \{0, 1, \dots, p-1\}$, where p is prime. Consider an element $x = \langle x_0, x_1, \dots, x_{n-1} \rangle \in U$. For any n -tuple $a = \langle a_0, a_1, \dots, a_{n-1} \rangle \in U$, define the hash function h_a by

$$h_a(x) = \left(\sum_{j=0}^{n-1} a_j x_j \right) \bmod p.$$

Let $\mathcal{H} = \{h_a : a \in U\}$. Show that $\mathcal{H}$ is universal, but not 2-independent. (*Hint:* Find a key for which all hash functions in $\mathcal{H}$ produce the same value.)

c. Suppose that we modify $\mathcal{H}$ slightly from part (b): for any $a \in U$ and for any $b \in \mathbb{Z}_p$, define

$$h'_{ab}(x) = \left(\sum_{j=0}^{n-1} a_j x_j + b \right) \bmod p$$

and $\mathcal{H}' = \{h'_{ab} : a \in U \text{ and } b \in \mathbb{Z}_p\}$. Argue that $\mathcal{H}'$ is 2-independent. (*Hint:* Consider fixed n -tuples $x \in U$ and $y \in U$, with $x_i \neq y_i$ for some i .)

What happens to $h'_{ab}(x)$ and $h'_{ab}(y)$ as a_i and b range over $\mathbb{Z}_p$?)

- d.* Alice and Bob secretly agree on a hash function h from a 2-independent family $\mathcal{H}$ of hash functions. Each $h \in \mathcal{H}$ maps from a universe of keys U to $\mathbb{Z}_p$, where p is prime. Later, Alice sends a message m to Bob over the internet, where $m \in U$. She authenticates this message to Bob by also sending an authentication tag $t = h(m)$, and Bob checks that the pair (m, t) he receives indeed satisfies $t = h(m)$. Suppose that an adversary intercepts (m, t) en route and tries to fool Bob by replacing the pair (m, t) with a different pair (m', t') . Argue that the probability that the adversary succeeds in fooling Bob into accepting (m', t') is at most $1/p$, no matter how much computing power the adversary has, even if the adversary knows the family $\mathcal{H}$ of hash functions used.

Chapter notes

The books by Knuth [261] and Gonnet and Baeza-Yates [193] are excellent references for the analysis of hashing algorithms. Knuth credits H. P. Luhn (1953) for inventing hash tables, along with the chaining method for resolving collisions. At about the same time, G. M. Amdahl originated the idea of open addressing. The notion of a random oracle was introduced by Bellare et al. [43]. Carter and Wegman [80] introduced the notion of universal families of hash functions in 1979.

Dietzfelbinger et al. [113] invented the multiply-shift hash function and gave a proof of Theorem 11.5. Thorup [437] provides extensions and additional analysis. Thorup [438] gives a simple proof that linear probing with 5-independent hashing takes constant expected time per operation. Thorup also describes the method for deletion in a hash table using linear probing.

Fredman, Komlós, and Szemerédi [154] developed a perfect hashing scheme for static sets—“perfect” because all collisions are avoided. An extension of their method to dynamic sets, handling insertions and

deletions in amortized expected time $O(1)$, has been given by Dietzfelbinger et al. [114].

The wee hash function is based on the RC6 encryption algorithm [379]. Leiserson et al. [292] propose an “RC6MIX” function that is essentially the same as the wee hash function. They give experimental evidence that it has good randomness, and they also give a “DOTMIX” function for dealing with variable-length inputs. Bellare et al. [42] provide an analysis of the security of the cipher-block-chaining message authentication code. This analysis implies that the wee hash function has the desired pseudorandomness properties.

¹ The definition of “average-case” requires care—are we assuming an input distribution over the keys, or are we randomizing the choice of hash function itself? We’ll consider both approaches, but with an emphasis on the use of a randomly chosen hash function.

² In the literature, a (c/m) -universal hash function is sometimes called c -universal or c -approximately universal. We’ll stick with the notation (c/m) -universal.